UNITÀ 3 · APPLICAZIONI JAVA PER LA COMUNICAZIONE IN RETE

Simulazione della verifica Unità 3

Stessa struttura e stessa difficoltà della verifica vera, con domande diverse. Fai la prova in 75 minuti senza guardare gli appunti, poi correggiti con le soluzioni in fondo.

Nome e cognome | Classe | Data

1. Scelta multipla

24 punti · 3 per domanda

Segna una sola risposta per ogni domanda.

a) Il campo TTL di un pacchetto IP…

- O misura il tempo di andata e ritorno
- O viene diminuito di 1 da ogni router
- O indica la porta di destinazione
- O conta i pacchetti persi

b) Con i privilegi necessari, `isReachable` invia…

- O un ICMP Echo Request
- O una richiesta HTTP HEAD
- O un SYN alla porta 80
- O un datagramma UDP alla porta 7

c) 8 pacchetti inviati, 6 risposte. La perdita è…

- O 6%
- O 2%
- O 25%
- O 75%

d) La banda di una connessione misura…

- O il tempo di risposta di un server
- O quanto varia la latenza
- O i salti tra i router
- O quanti bit al secondo possono passare

e) `Future.get()` …

- O aspetta la fine del compito e ne restituisce il risultato
- O lancia il compito in un nuovo thread
- O annulla il compito
- O restituisce subito null se il compito non è finito

f) Per servire più client TCP contemporaneamente, il server di chat…

- O apre un ServerSocket per ogni client
- O usa UDP invece di TCP
- O avvia un thread per ogni client accettato
- O chiama accept() solo una volta

g) Nell'handshake WebSocket, il browser manda un valore casuale nell'header…

- O Sec-WebSocket-Accept
- O Sec-WebSocket-Key
- O Origin
- O Upgrade

h) I Server-Sent Events trasportano dati…

- O nei due sensi
- O solo dal browser al server
- O solo tra due browser
- O solo dal server al browser

2. Vero o falso

12 punti · 1,5 per frase

V ☐ **F** ☐ Il RTT misura il tempo di sola andata di un pacchetto.

V ☐ **F** ☐ Nel tentativo TCP di `isReachable`, anche una connessione rifiutata (RST) indica che qualcuno ha risposto.

V ☐ **F** ☐ Costruire il comando con `"cmd /c ping " + host` espone a command injection.

V ☐ **F** ☐ Un compito I/O-bound passa la maggior parte del tempo ad aspettare.

V ☐ **F** ☐ Con i virtual thread di Java 21, `isReachable` esegue sempre tutti i controlli in parallelo.

V ☐ **F** ☐ TCP garantisce che a ogni `println` del server corrisponda esattamente un `read` del client.

V ☐ **F** ☐ Un frame WebSocket di tipo ping ha l'opcode 0x9.

V ☐ **F** ☐ `ws://` cifra i dati con TLS.

3. Leggi l'output del ping

10 punti

```
64 bytes from 10.1.1.1: icmp_seq=1 ttl=62 time=30.0 ms
64 bytes from 10.1.1.1: icmp_seq=2 ttl=62 time=34.0 ms
64 bytes from 10.1.1.1: icmp_seq=3 ttl=62 time=31.0 ms
64 bytes from 10.1.1.1: icmp_seq=5 ttl=62 time=39.0 ms
--- 10.1.1.1 ping statistics ---
5 packets transmitted, 4 received
```

a) Percentuale di pacchetti persi, e quale numero di sequenza manca (2 p.)

_____

b) Minimo, massimo e RTT medio (3 p.) _____

c) Jitter, come media delle differenze tra RTT consecutivi ricevuti, con il calcolo (3 p.)

_____

d) Il TTL vale 62. Se l'host è partito da 64, quanti router ci sono in mezzo? (2 p.) _____

4. Quanto dura la scansione?

6 punti · 2 per caso

120 host tutti spenti, timeout di 2 secondi per host. Scrivi il calcolo e il tempo totale approssimato.

a) In sequenza: _____ b) Pool di 40 thread: _____ c) Pool di 25 thread: _____

5. Trova l'errore

8 punti

Uno studente vuole contare quanti host rispondono:

```
static int vivi = 0;
...
try (ExecutorService pool = Executors.newFixedThreadPool(32)) {
    for (String h : hosts)
        pool.submit(() -> { if (controlla(h).vivo()) vivi++; });
}
System.out.println("Host vivi: " + vivi);
```

a) Perché il risultato può essere sbagliato, e perché l'errore non si vede sempre? (4 p.)

b) Proponi due correzioni diverse, scrivendo le righe che cambiano. (4 p.)

6. Completa il server eco e il client

10 punti · 1 per spazio

```
try (ServerSocket server = new __________(5000)) {                          // (1)
    while (true) {
        try (Socket s = server.accept();
             var in = new BufferedReader(new InputStreamReader(s.__________(), UTF_8));   // (2)
             var out = new PrintWriter(s.__________(), true, UTF_8)) {                    // (3)
            String riga;
            while ((riga = in.readLine()) != null)
                out.println("ECO " + riga.__________());                     // (4) in maiuscolo
        }
    }
}
// client
try (Socket s = new Socket("127.0.0.1", ______)) {                          // (5)
    Thread.__________().start(() -> { /* stampa le righe del server */ });     // (6) Java 21
    ...
}
// eccezione se nessun server ascolta sulla porta: __________Exception          // (7)
// eccezione se la porta è già occupata da un altro server: __________Exception // (8)
// il valore restituito da readLine() per una riga vuota: ______               // (9)
// la classe Java per un server TLS: __________                                 // (10)
```

7. Il protocollo in azione

10 punti · 2,5 per evento

Il server usa il protocollo del laboratorio (`NICK`, `MSG`, `QUIT`; risposte `OK`, `ERR`, `MSG`, `INFO`). Sono collegati anna e bruno. Per ogni evento scrivi chi riceve qualcosa e quale riga esatta.

Evento	Chi riceve e che cosa
1. Un nuovo client scrive `NICK carla`	
2. bruno scrive `MSG ci sono`	
3. carla scrive `msg ciao` (in minuscolo)	
4. anna scrive `QUIT`	

8. Un handshake da completare

10 punti

```
GET /chat/ws HTTP/1.1
Host: localhost:8080
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

a) Scrivi la prima riga della risposta del server e i due header che confermano il cambio di protocollo. (3 p.)

b) Scrivi in Java il metodo `accept(String key)` che calcola Sec-WebSocket-Accept (la stringa fissa è `258EAFA5-E914-47DA-95CA-C5AB0DC85B11`). (4 p.)

c) Dopo l'handshake il browser manda un messaggio di testo e poi chiude la scheda. Quali opcode hanno i due frame? (3 p.)

9. Progetto

10 punti

Un'azienda di consegne vuole: (A) un monitor dei suoi **30 server** con un allarme quando uno cambia stato; (B) una pagina in cui i clienti vedono **lo stato del proprio ordine** aggiornarsi da solo (preparato, partito, consegnato). Descrivi l'architettura e le tecnologie per A e per B, motivando ogni scelta con un confronto e indicando almeno due misure di sicurezza.

Punti: _____ / 100

Voto: _____

Soluzioni

1. a) viene diminuito di 1 da ogni router · b) un ICMP Echo Request · c) 25% (2 su 8) · d) quanti bit al secondo possono passare · e) aspetta la fine del compito e ne restituisce il risultato · f) avvia un thread per ogni client accettato · g) Sec-WebSocket-Key · h) solo dal server al browser.

2. F (è andata e ritorno) · V · V (un host come `8.8.8.8 & del *.*` diventa un secondo comando) · V · F (isReachable è una chiamata nativa che blocca il thread portatore: al massimo tanti controlli quanti i core) · F (TCP è un flusso di byte, i confini li decide il protocollo) · V · F (cifra solo `wss://`).

3. a) 1 su 5 = 20%, manca icmp_seq=4 · b) minimo 30 ms, massimo 39 ms, media (30+34+31+39)/4 = 134/4 = 33,5 ms · c) |34−30| + |31−34| + |39−31| = 4 + 3 + 8 = 15; 15/3 = 5 ms · d) 64 − 62 = 2 router.

4. a) 120 × 2 s = 240 s · b) 120/40 = 3 turni × 2 s = 6 s · c) ⌈120/25⌉ = 5 turni × 2 s = 10 s (l'ultimo turno incompleto dura comunque un timeout).

5. a) `vivi++` non è atomico (leggi, somma, scrivi): due thread possono leggere lo stesso valore e un incremento va perso. È una race condition: dipende dai tempi, quindi a volte il risultato è giusto e a volte no. b) Correzioni: `static AtomicInteger vivi` con `vivi.incrementAndGet()`; oppure raccogliere i `Future<Esito>` in una lista e contare nel main dopo i `get()` (niente variabili condivise); oppure un blocco `synchronized` attorno all'incremento.

6. (1) ServerSocket · (2) getInputStream · (3) getOutputStream · (4) toUpperCase · (5) 5000 · (6) ofVirtual · (7) Connect (ConnectException, "Connection refused") · (8) Bind (BindException, "Address already in use") · (9) "" (stringa vuota, non null) · (10) SSLServerSocket.

7. 1) carla riceve `OK benvenuto carla`; anna e bruno ricevono `INFO carla entra nella chat` · 2) anna e carla ricevono `MSG bruno ci sono` · 3) solo carla riceve `ERR comando sconosciuto` (il protocollo distingue le maiuscole) · 4) bruno e carla ricevono `INFO anna esce dalla chat`; anna non riceve nulla e la connessione si chiude.

8. a) `HTTP/1.1 101 Switching Protocols`, `Upgrade: websocket`, `Connection: Upgrade` (più `Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=`, l'esempio della RFC 6455).

```
static String accept(String key) throws Exception {
    byte[] sha1 = MessageDigest.getInstance("SHA-1")
        .digest((key + "258EAFA5-E914-47DA-95CA-C5AB0DC85B11").getBytes(StandardCharsets.UTF_8));
    return Base64.getEncoder().encodeToString(sha1);
}
```

c) testo: opcode **0x1**; chiusura: opcode **0x8** (con un codice come 1001, "la pagina se ne va").

9. Esempio di risposta. A: monitor in Java con controlli TCP sulle porte dei servizi (funzionano anche dove ICMP è filtrato e non servono privilegi), timeout 2 s, virtual thread (i socket non bloccano i thread veri): caso peggiore circa 2 s per giro; ScheduledExecutorService ogni 30 s, stato precedente in una ConcurrentHashMap e allarme solo al cambio di stato (email o messaggio al tecnico), storico in un CSV o in un database. B: Server-Sent Events, perché gli aggiornamenti vanno solo dal server al cliente e sono rari; WebSocket funzionerebbe ma è più complesso del necessario, il polling ogni pochi secondi sprecherebbe richieste. Sicurezza: HTTPS (e quindi SSE cifrati), ogni cliente vede solo il proprio ordine (autenticazione o link con un codice non indovinabile), monitor accessibile solo dalla rete interna.